

NANYANG TECHNOLOGICAL UNIVERSITY

Example Class

Prof Douglas Maskell

asdouglas@ntu.edu.sg

N4-02c-98

School of Computer Engineering

NTU

Course Site: https://ntulearn.ntu.edu.sg/

Schedule

Week	Topic	Tutorial	Lecture 1 (Monday 8:30)	Lecture 2 (Wednesday 8:30)
1	Mod 1 (Intro, Review Verilog, Design Flow, IP)	No Tut	Introduction	Not used
2	Mod 2 (Verification & Verilog Testbenches)	Tut 1	Not Used	Not used
3	Mod 3 (FPGA Architecture and Synthesis)	Tut 2	Not used	Not Used
4	Mod 4 (Arithmetic on FPGAs)	Tut 3	Not used	**EC1** (Mod 1 & 2)
5	Mod 5 (Timing and Pipelining)	Tut 4	Not Used	**EC2** (Mod 3 & 4)
6	Mod 5 (Timing and Pipelining) continued	Tut 5	Not used	Not used
7	Mod 7 (**eLearning** Buses & interfacing)	No Tut	Not used	**QUIZ**
Recess				
8	Mod 8 (uP Case Study)	No Tut	Not Used	**EC3** (Mod 5 & 7)
9	Mod 8 (uP Case Study)	Tut 7	Not used	Not used
10	Mod 9 (Asynchronous Design)	Tut 8	Not used	**EC4** (Mod 8)
11	Mod 9 (Asynchronous Design)	Tut 9	Not Used	Not used
12	--	Tut 10	Not used	**EC5** (Mod 9)
13	--	Not used	Not used	Not used

The yellow shaded regions are for Example Classes (which are in addition to the LAMS-based lectures).
The green shaded region is for a class quiz (details will be provided later).
There is no Tutorial 6 as this has been integrated into the eLearning module.
The current Lab schedule can be accessed from: https://freeaccess.scse.ntu.edu.sg/scselabs

This may change slightly throughout the semester
Individual Tutors are likely to change the tutorial timing.



NANYANG TECHNOLOGICAL UNIVERSITY

Example Class

Module 5

Timing and pipelining

Module 7

Buses and Interfacing

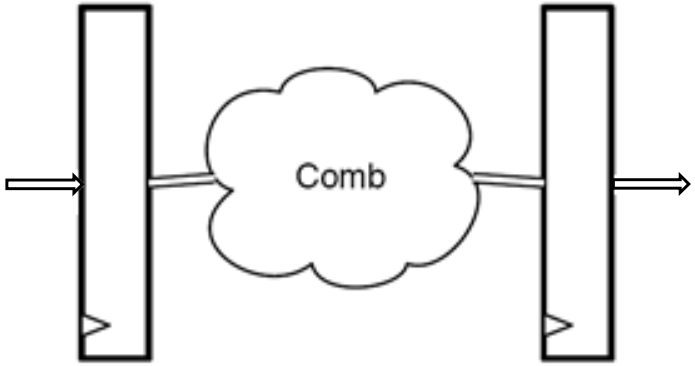
Summary (Mod 5)

- Timing and Pipelining
  - Timing in gate level combinational circuits
    - Introduced the concept of the Critical Path
  - Added registers at inputs and outputs
    - Allowed us to determine how fast we can clock the circuit
  - Introduced Throughput & Latency
  - Looked at Pipelining
    - Added additional registers to break long combinational paths
  - Data paths
    - ASAP & ALAP Scheduling
    - Consequence of imbalanced pipeline stages

Timing Basics

- The output of a gate will change a short period after the inputs are changed
- Signals take time to move through wires
- Typically, we are concerned with the worst-case circuit delay (the critical path). This then determines how fast we can operate the circuit
- Different path delays can cause internal glitches, where a signal changes momentarily, before settling to the final (stable) value

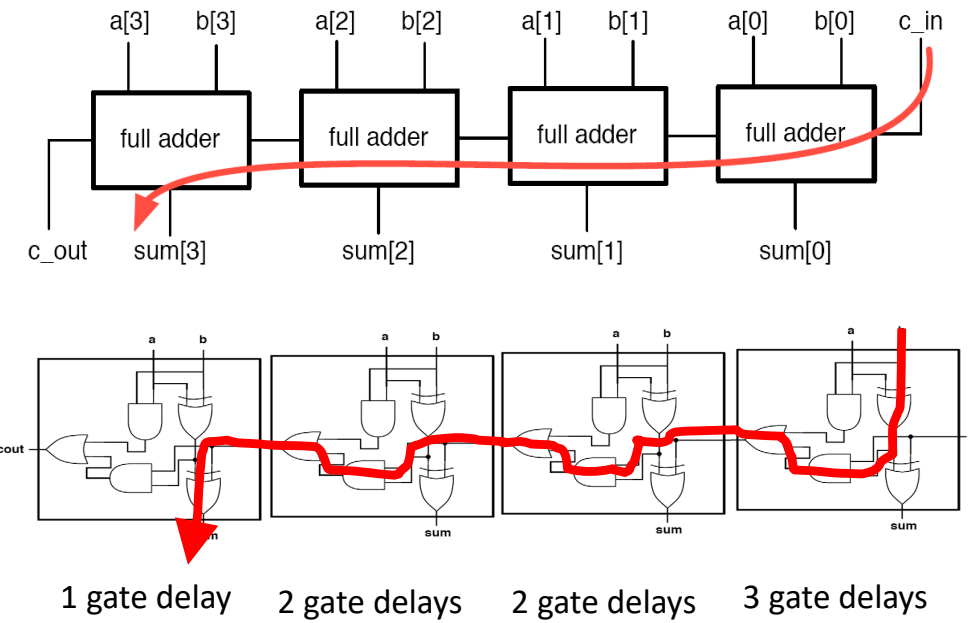
- In FPGAs
  - we don't care about gate delays, as logic is implemented in LUTs, not gates
    - A 6-input LUT can realise any 6-input function (or 2 functions of the same 5 inputs, etc)
    - The propagation delay (through the LUT) is the same no matter what function is implemented
    - If a function is too large for a single LUT, it can be spread across multiple LUTs
    - Routing delay is also significant. Depends on how the components are connected



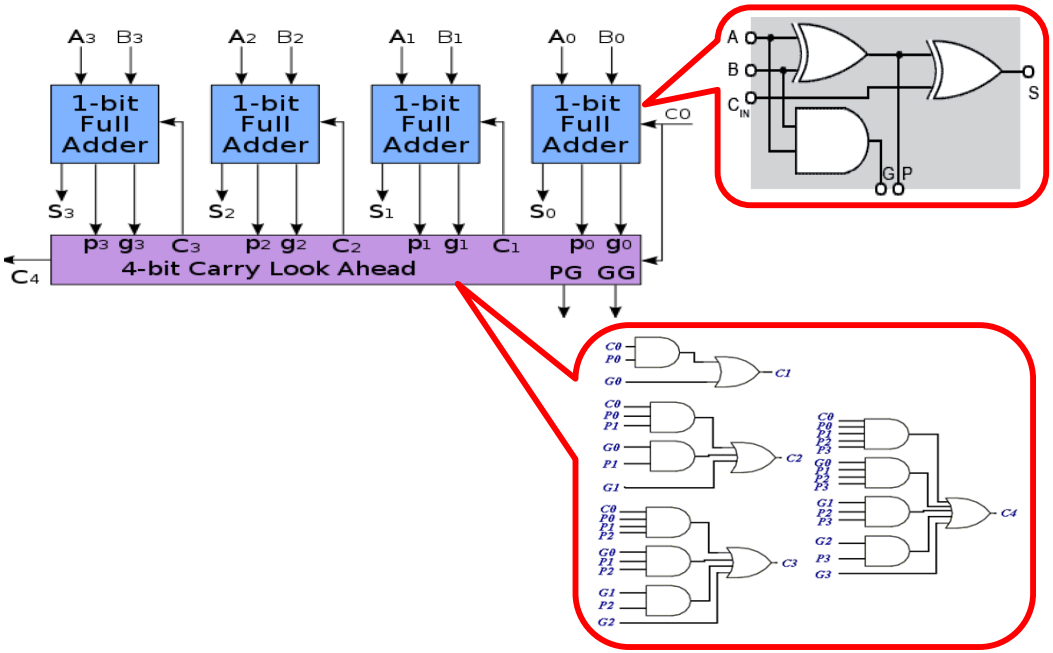
We also looked at adding registers before and after the circuit. The critical path determined how fast we could clock the circuit.
Also looked at the impact of the timing characteristics of the registers (t_{cQ} t_s and t_h)

Adder delay revisited

Ripple-carry adders (RCA)



Carry look-ahead adders (CLA)



Total delay is 8 gate delays to sum3.
2n gate delays (n is bit size of the adder)

Delay from A_0 / B_0 to S_3 is: 1 in FA, 2 in CLA unit & 1 in last FA (FA_3) = **4 gate delays**

16-bit adders: the RCA delay is 32 gate delays, while it is just 8 gate delays for CLA
This means that if a gate delay is 1ns, we can clock the CLA at 1/8ns = 125MHz, while we can only clock the RCA at 1/32ns = 31.25MH. **A significant difference.**

Synchronous systems

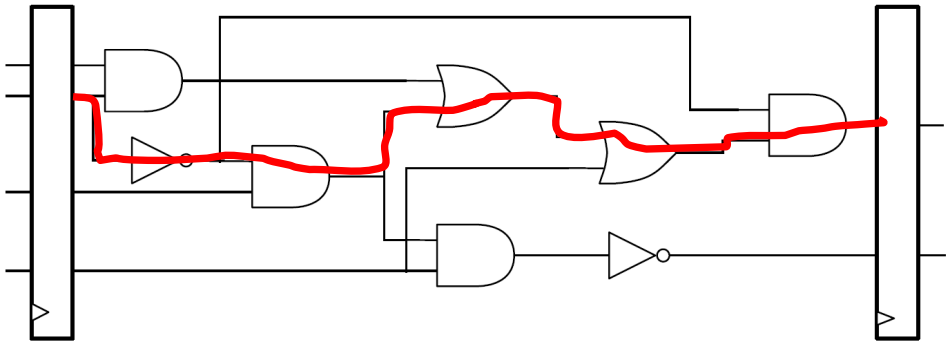
- Registers give us a predictable, fixed rate of data movement, and help us model the system in a more understandable way
- For synchronous systems, there are two main timing metrics:
  - Throughput:
    - The number of input values that can be processed per unit time.
  - Latency:
    - The time between an input entering the system, and its corresponding output emerging from the system

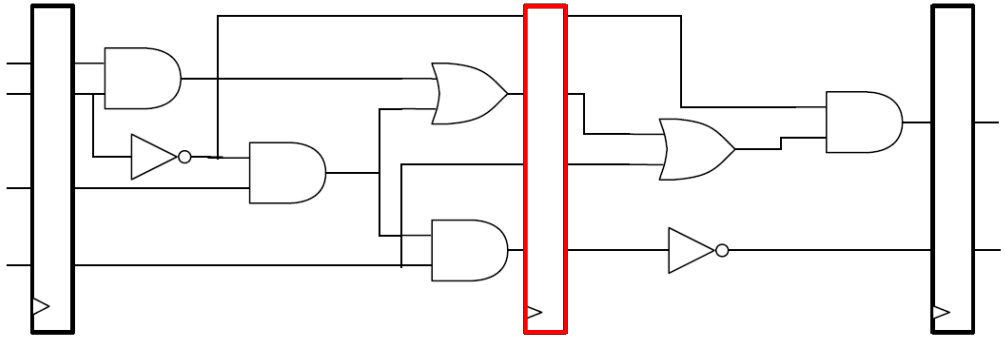
When building circuits, we would like to maximise **Throughput** and minimise **Latency** (but this is usually conflicting)

Generally, we get better results by maximising throughput as much as we can, while tolerating a small increase in latency.

Pipelining

- The largest chunk of combinational logic between any two registers in our design will determine our maximum frequency
- To increase throughput, we can break this logic down by adding another register in the middle. Now our largest chunk is smaller, and we can clock the circuit faster. But we add extra latency.





Assume a 1ns gate delay, then can clock the circuit at max 1/5ns = **200MHz**.
The latency is one clock cycle (**5ns**) from input to result (ignoring t_{cQ} and t_s)

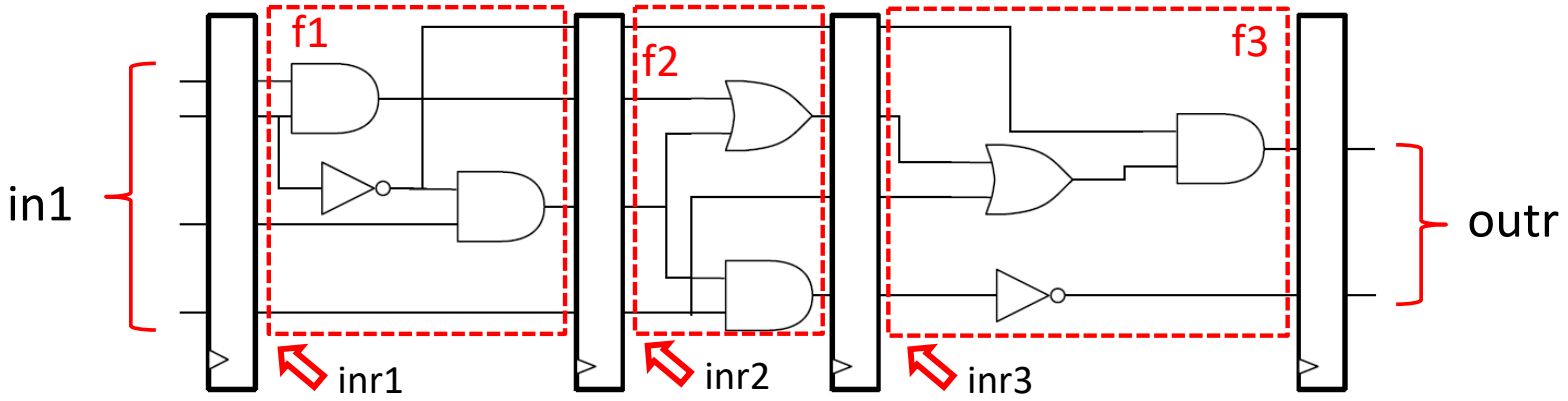
We can improve throughput by adding a register stage in the middle .
Critical path is now 3 gate delays. Can clock the circuit at 1/3ns = **333MHz**.
The latency is 2 clock cycles (**6ns**)

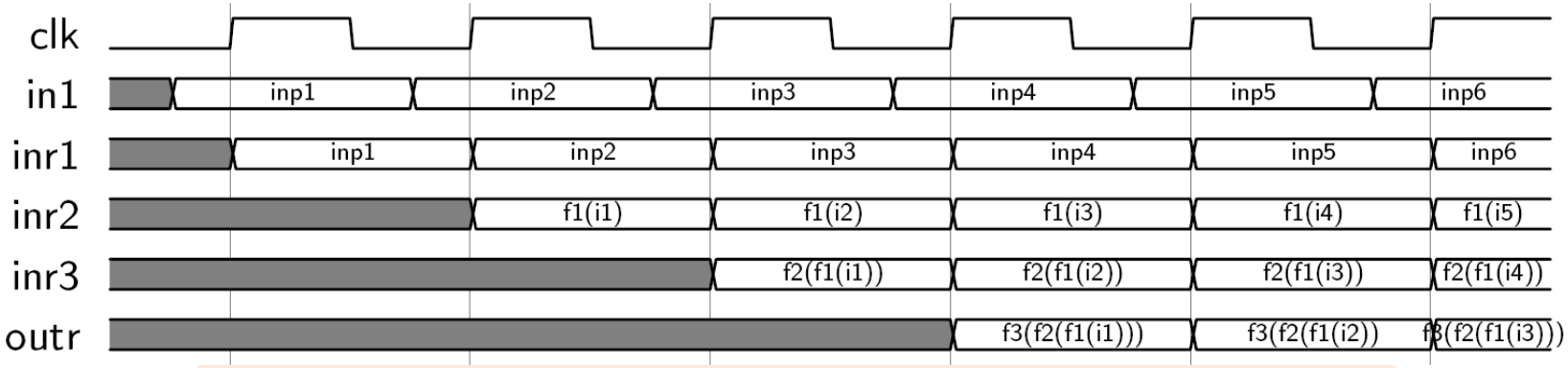
Note: we must cut ALL signal paths.

Pipelining

– We can add more stages. Adding a 3rd stage gives a critical path = 2ns; frequency = 500MHz and latency = 6ns (no change from prev.).

(Note: original was 200MHz and 5ns)





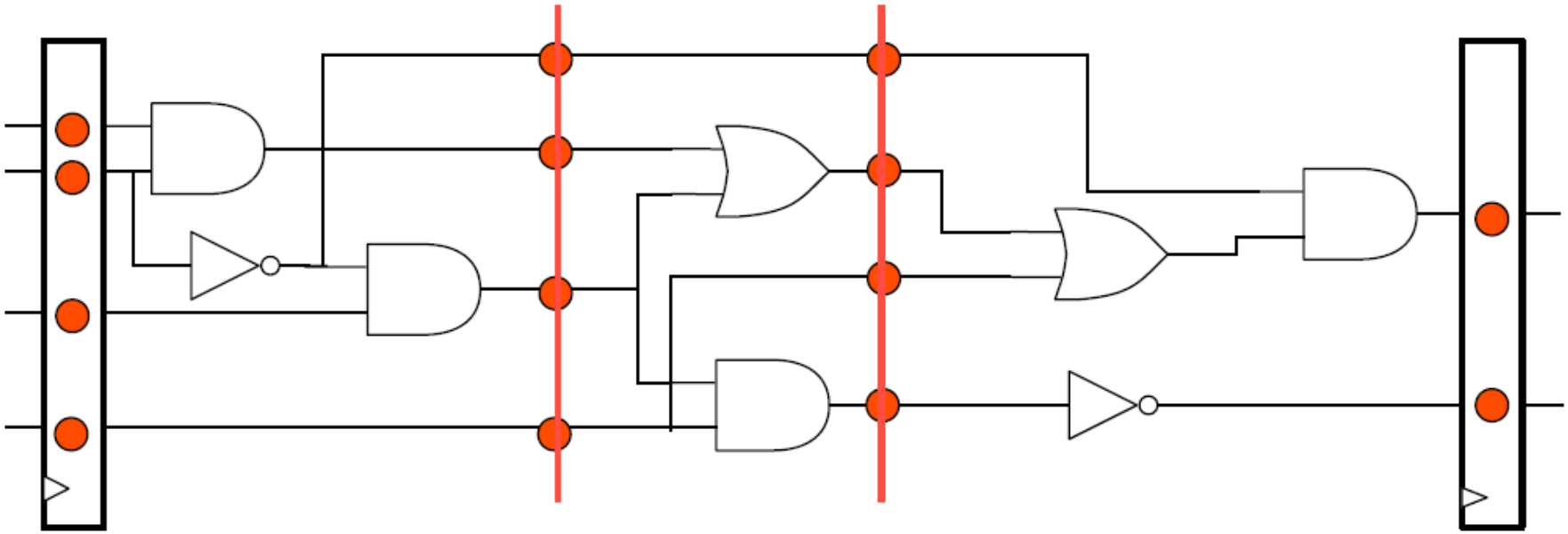
Note the signals after the registers are now aligned to the clock

So pipelining can increase the clock frequency by adding register stages to split up the combinational logic

Pipelining

One key rule:

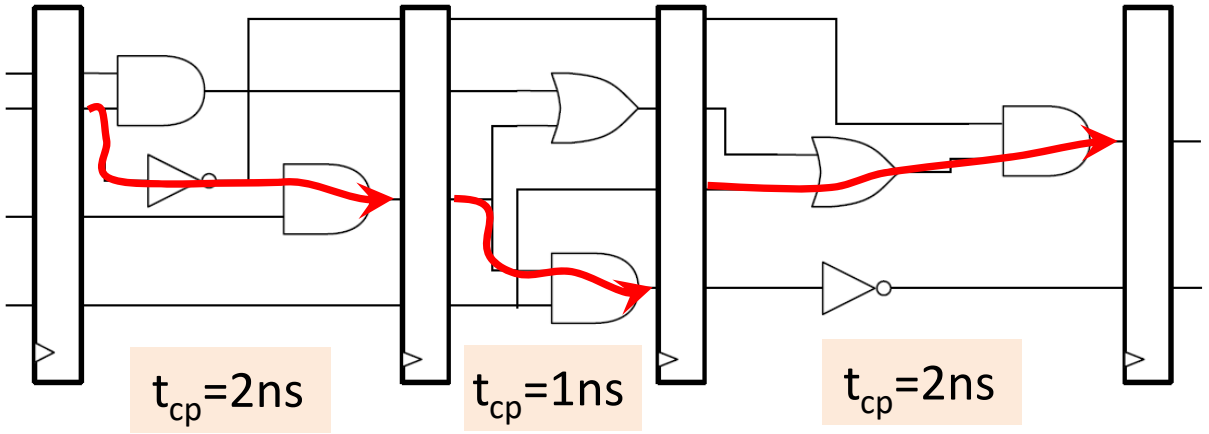
Always add registers to **all paths** that you cross when adding a pipeline stage! And make sure you cross ALL paths.



The best strategy is to draw lines that cross all paths from one register to the next – where they intersect a signal, a register is needed

Pipelining

It is important to also try to balance the pipeline stages

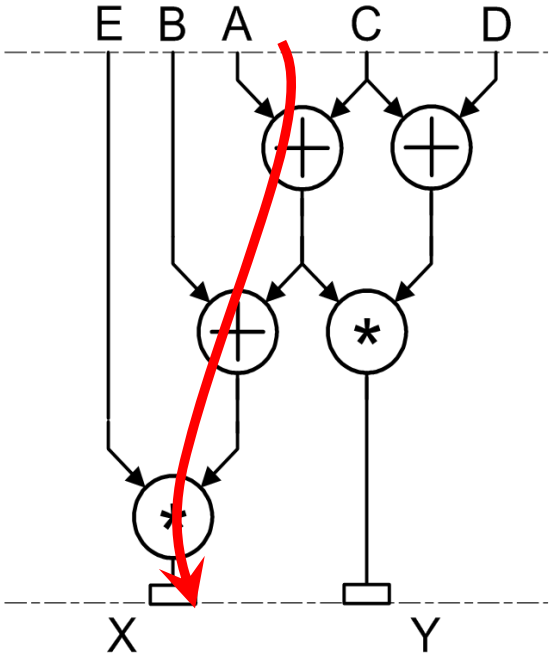


Here the middle stage is being limited by the worst-case critical path (2ns)

Pipelining

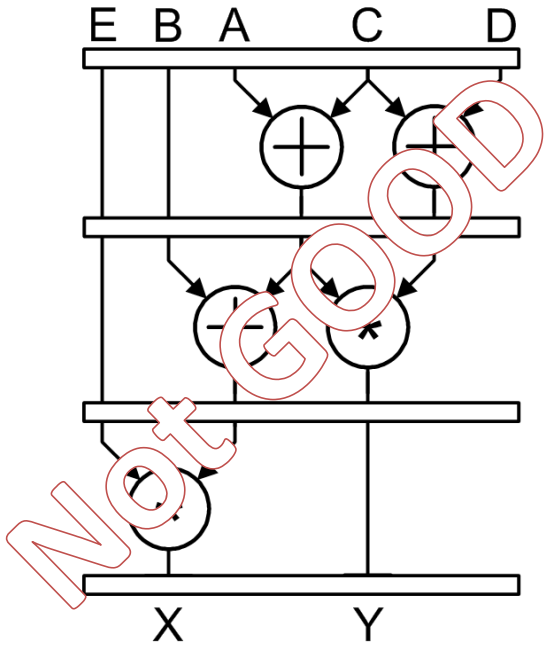
It is important to also try to balance the pipeline stages

- Consider



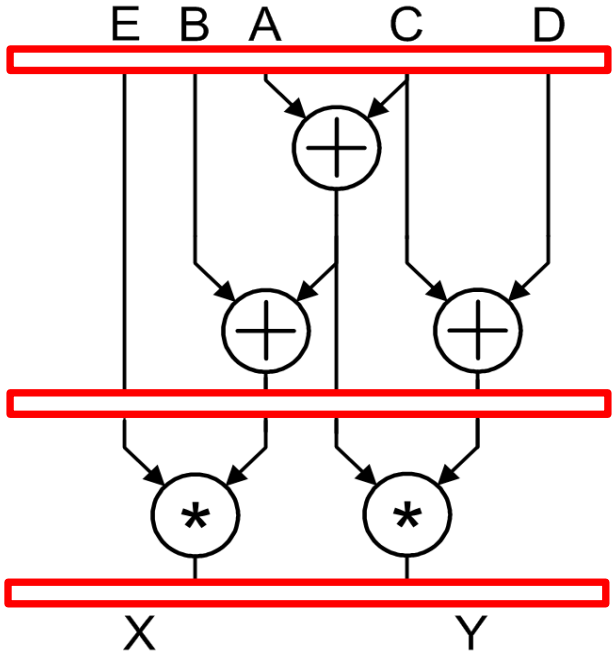
An adder has a 1ns delay, and a multiplier has a 5ns delay. Can clock the circuit at: 1/7ns = **142MHz**. Latency = 7ns.

Pipelined (POOR)



Then, we can clock the circuit at: 1/5ns = **200MHz**. Latency = 15ns.

Better Pipelining



Using just 2 pipeline stages, we can clock the circuit at: 1/5ns = **200MHz**. (same) Latency = 10ns. **Much better!**

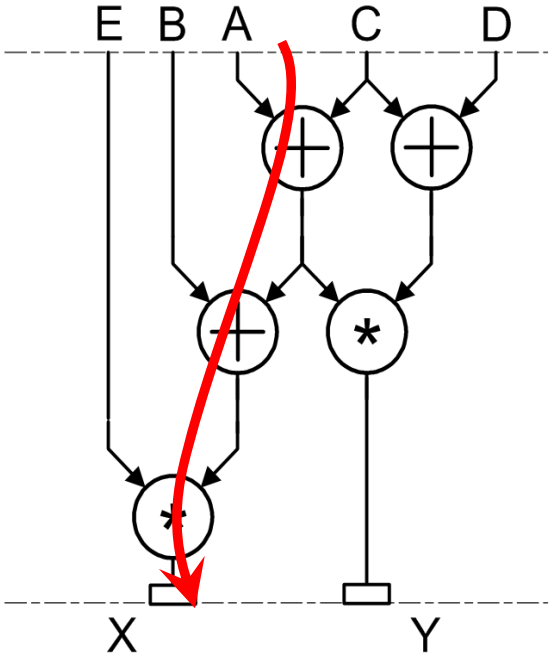
Pipelining

It is important to a

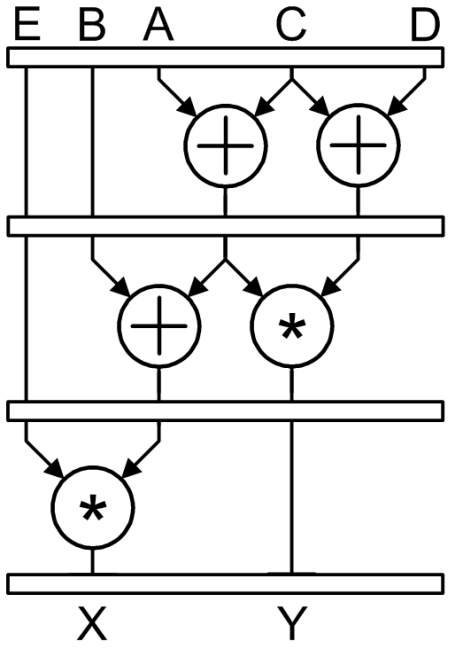
Thus it is very important to try to balance the pipeline stages

stages

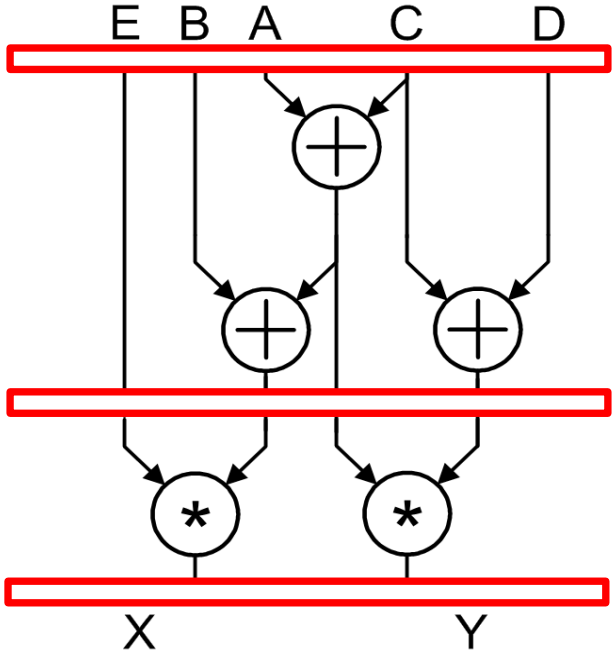
- Consider



An adder has a 1ns delay, and a multiplier has a 5ns delay. Can clock the circuit at: 1/7ns = **142MHz**. Latency = 7ns.



Then, we can clock the circuit at: 1/5ns = **200MHz**. Latency = 15ns.



Using just 2 pipeline stages, we can clock the circuit at: 1/5ns = **200MHz**. Latency = 10ns. Much better!

Summary (Mod 7)

- Buses and Interfacing
  - PCI express
  - Buses, ending with network on chip

Questions from Lectures

Q1 (a) A Gen2 PCIe has a transfer rate of up to 5Gb/s and uses an 8b/10b encoding scheme. (Lower case "b" means bits not Bytes)

Thus a single lane operating in 1 direction has a peak throughput of:

5Gbits/s x 8/10 x 1/8 = **500MB/s**

4 separate lanes allow a peak transfer rate of 2GB/s

Q1 (b) Gen3 PCIe has a transfer rate of up to 8Gb/s and uses a 128b/130b encoding scheme.

Thus a single lane operating in 1 direction has a peak throughput of:

8Gbits/s x 128/130 x 1/8 = **986.6MB/s ≈ 1GB/s**

4 separate lanes allows a peak transfer rate of ~4GB/s

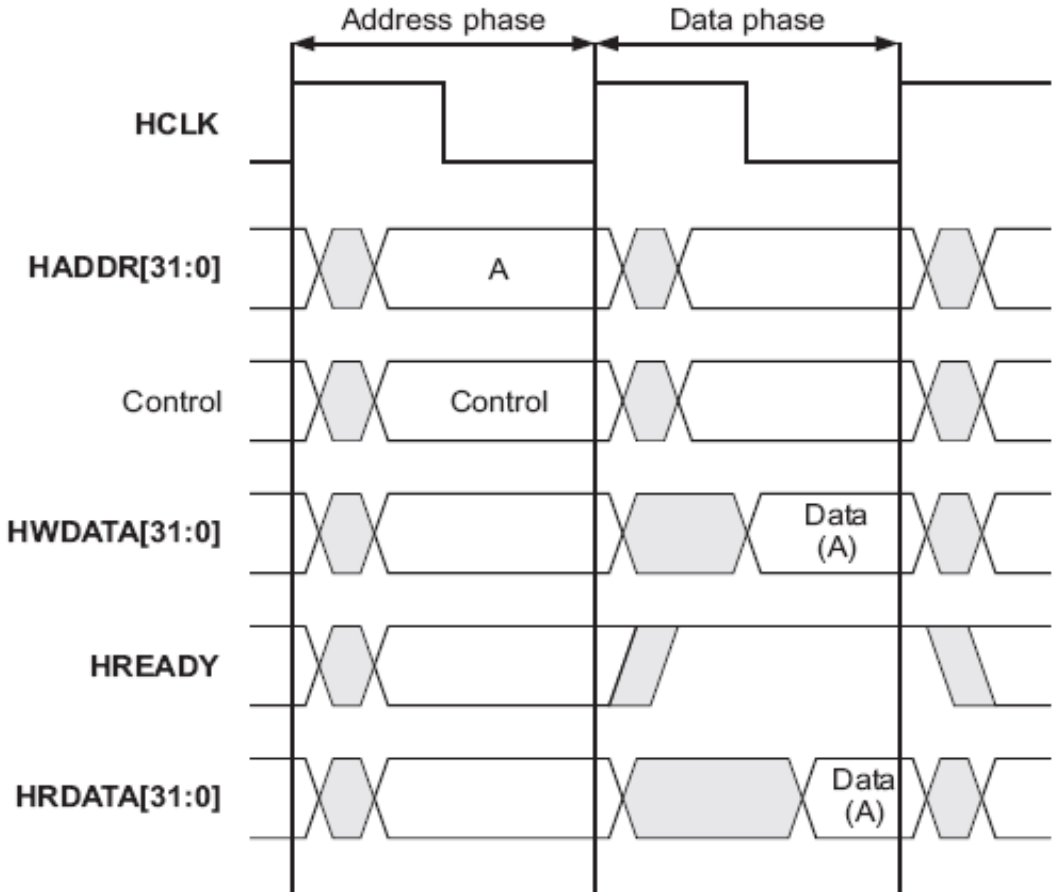
Q1 (c) Thus an increase in bit rate from 5Gb/s to 8Gb/s and a change in the encoding scheme has resulted in a doubling of the peak transfer rate.

Questions from Lectures

Q2 Calculate the peak bandwidth of a 200MHz 32 bit AMBA ABH bus

(a) The ABH **basic transfer** has two repeating cycles, The address phase and the data phase, each of which takes 1 cycle.

Peak bandwidth = 1/2 x 200M x 32 x 1/8 = **0.4GB/s**



Questions from Lectures

Q2 Calculate the peak bandwidth of a 200MHz 32 bit AMBA ABH bus

(a) The ABH **basic transfer** has two repeating cycles, The address phase and the data phase, each of which takes 1 cycle.

Peak bandwidth = 1/2 x 200M x 32 x 1/8 = **0.4GB/s**

(b) The ABH basic transfer has two repeating cycles, but the data phase has a **wait state** and takes 2 cycles

Peak bandwidth = 1/3 x 200M x 32 x 1/8 = **0.2666GB/s**

(c) A 16 beat **pipelined burst** transfer has 1 address cycle and 16 data cycles, thus 16 out of 17 cycles are dedicated to data transfer

Peak bandwidth = 16/17 x 200M x 32 x 1/8 = **0.7529GB/s**

(d) A 16 beat pipelined burst transfer with **1 wait state** has 1 address cycle and 16 (2 cycle) data cycles, resulting in 16 data transfers in 33 cycles.

Peak bandwidth = 16/33 x 200M x 32 x 1/8 = **0.3879GB/s**

Discussion Question

Why are bus topologies not the preferred solution for connecting the cores in a many core (1000 core) processor?

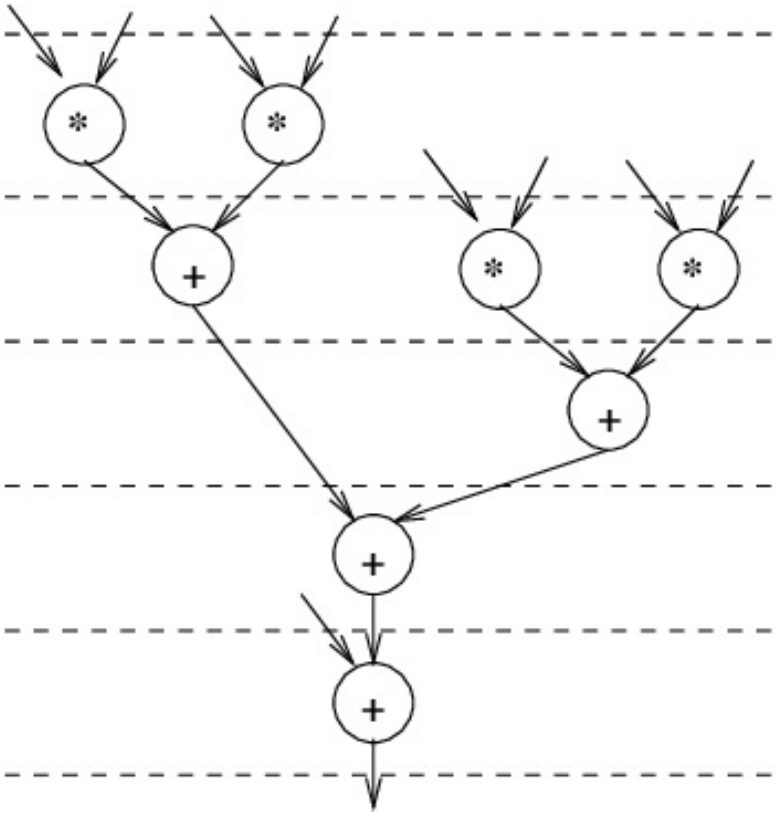
ANS:

- Speed becomes a major problem as more devices are added to a bus. The capacitance grows and hence the RC time constant increases causing a slowdown in signal propagation over the bus. The latency (and the Arbitration latency) increase. As there are potentially more devices wanting to be the bus master, arbitration becomes a slower and more difficult problem
- Buses do not provide much in the way of concurrency. Unless you partition the bus, it is not possible to have multiple transfers at the same time. As the bandwidth must be shared, communications becomes a bottleneck.
- As the capacitance grows, so does the power. In large bus structures, power becomes an issue.
- Buses can become a single point of failure

Exam Solutions

Q2 S2 18/19

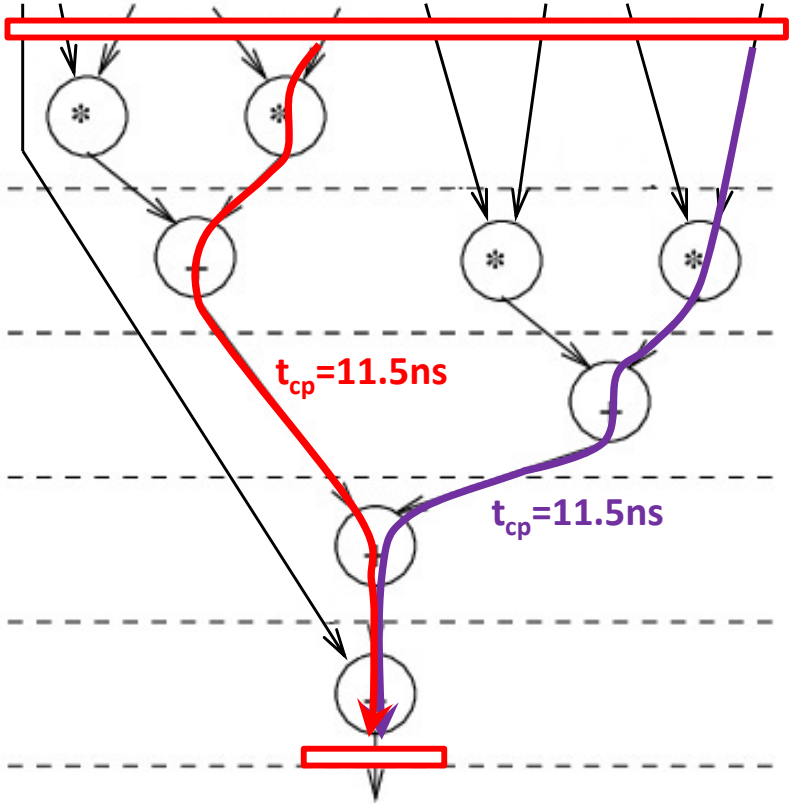
a) The data flow graph of a simple biquad filter is given. The inputs and output are registered. Given propagation delays: t_{add} = 2ns, t_{mult} = 5ns, t_{CQ} = 0.4ns and t_S = 0.1ns, determine the following:



(i) The critical path delay, maximum operating frequency, throughput and latency for the non-pipelined implementation.

Soln to Last Semester Exam

Q2 S2 18/19 a) The data flow graph of a simple biquad filter is given. The inputs and output are registered. Given propagation delays: t_{add} = 2ns, t_{mult} = 5ns, t_{CQ} = 0.4ns and t_S = 0.1ns, determine the following:



(i) The critical path delay, maximum operating frequency, throughput and latency for the non-pipelined implementation.

ans:

First we must extend inputs to the top and then add the input and output registers. Then:

Critical path delay: 0.4+5+2+2+2+0.1 = 11.5ns

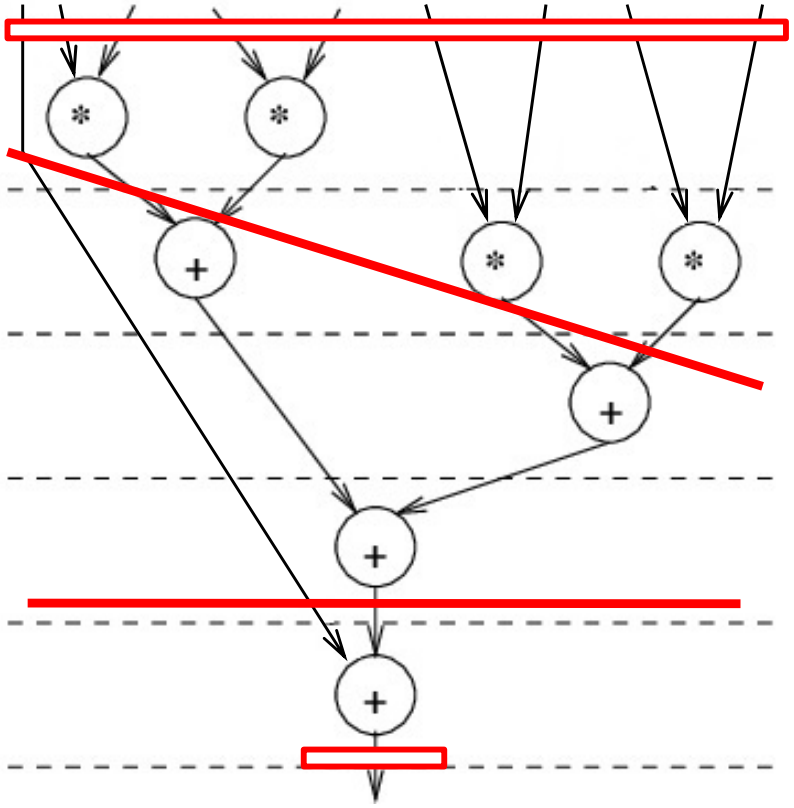
Max Frequency: 1/11.5ns = **86.956MHz**

Throughput: A sample every clk (every 11.5 ns)

Latency: **11.5ns**

Soln to Last Semester Exam

Q2 S2 18/19 a) The data flow graph of a simple biquad filter is given. The inputs and output are registered. Given propagation delays: t_{add} = 2ns, t_{mult} = 5ns, t_{CQ} = 0.4ns and t_S = 0.1ns, determine the following:



Note: the latency has increased (from 11.5ns) so in practice (not part of the question) you could trade off speed for latency. Two pipeline stages would give 6.5ns (153.85MHz), latency = 13ms.

(ii) The pipelined implementation which gives the highest frequency with the fewest pipeline stages. The multipliers and adders are NOT able to be internally pipelined. Sketch the circuit.

ans: First put all multipliers in one pipeline stage. Then, we have 6ns of adder critical path, so to achieve the highest frequency we must add an extra register stage between the adders. Total: 3 stages (**Note** that the lower internal register could be moved up)

(iii) The critical path delay, maximum frequency, throughput and latency for the pipelined implementation in Q2 a (ii).

ans:

Critical path delay: 0.4 + 5 + 0.1 = 5.5ns

Max Frequency: **181.818MHz**

Throughput: A sample every clk (every 5.5 ns)

Latency: **16.5ns**

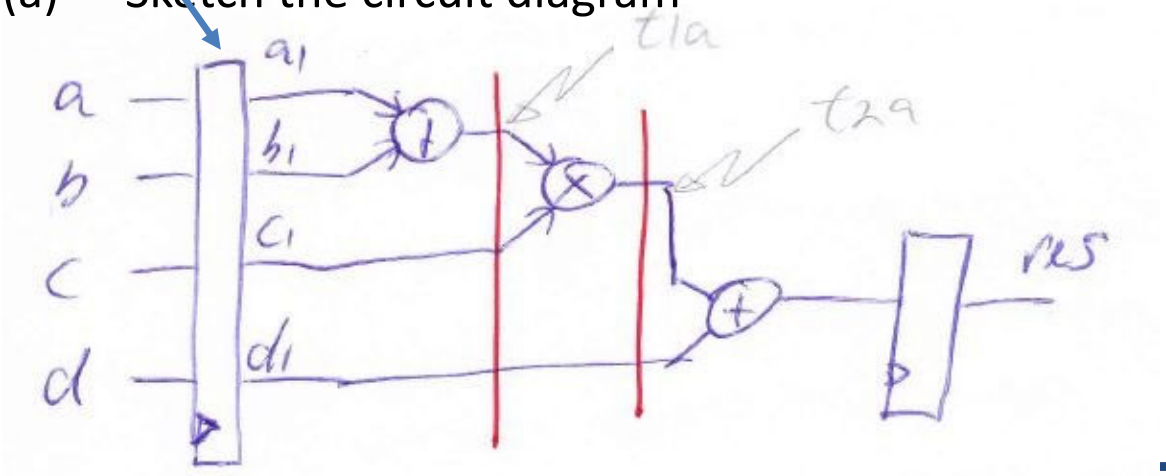
Soln to 17/18 Exam

Q2 S2 17/18 The Verilog code shown in Figure Q2 implements an arithmetic circuit, where *a*, *b* and *c* are 12-bit signed inputs, and *d* is a 24-bit signed input.

```
always @ (posedge clk) begin
    if (rst) begin
        a1 <= 0; b1 <= 0;
        c1 <= 0; d1 <= 0;
        res <= 0;
    end else begin
        a1 <= a; b1 <= b;
        c1 <= c; d1 <= d;
        res <= (a1 + b1) * c1 + d1;
    end
end
```

Note the register at the input

(a) Sketch the circuit diagram



Note: Without the red

(b) Modify to fully pipeline the **always** block. **(regs in red)**

```
always @ (posedge clk) begin
    if (rst) begin
        // SET a1, b1, c1, d1, t1a,
        // t1c, t1d, t2a, t2d, res to 0
    end else begin
        a1 <= a;  b1 <= b;
        c1 <= c;  d1 <= d;
        t1a <= a1+b1;
        t1c <= c1;
        t1d <= d1;
        t2a <= t1a * t1c;
        t2d <= t1d;
        res <= t2a + t2d;
    end
end
```

Soln to 17/18 Exam

Q2 S2 17/18

(c) If a multiplier has a propagation delay of 3ns, and an adder has a propagation delay of 1ns, determine the following:

(i) The maximum frequencies that the original and modified circuits can be clocked at.

ANS. original: 1/5ns = 200MHz, pipelined: 1/3ns = 333.3MHz

(ii) The latency of the fully pipelined implementation in Q2(b) in terms of the number of clock cycles and in nanoseconds from the moment the inputs are clocked into the input registers.

ANS. Latency is 3 clock cycles = 9ns

(iii) The maximum frequency for the fully pipelined implementation if the flip-flop characteristics are: clock-to-Q time (t_{cQ}) is 0.6ns, setup time (t_s) is 0.3ns, and hold time (t_h) is 0.1ns.

ANS. The critical path is: $t_{CQ} + t_{mult} + t_S$ = 0.6+3.0+0.3 = 3.9ns

Thus, the maximum frequency is 1/3.9ns = 256.4MHz

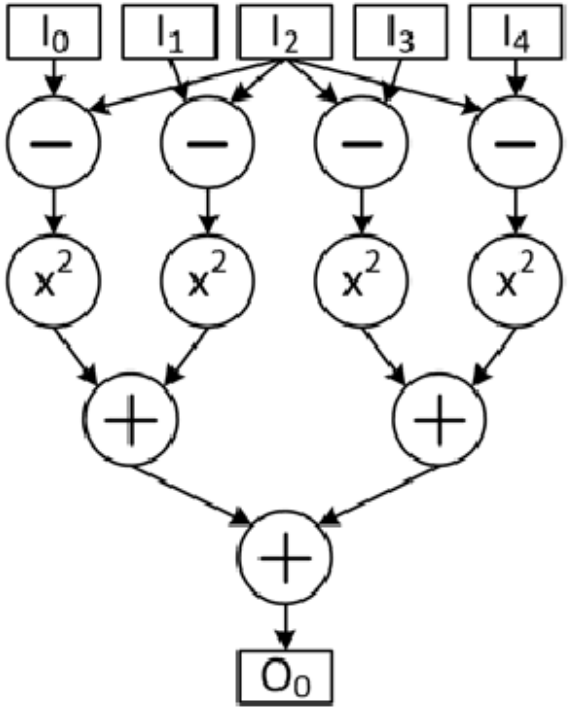
Note: The hold time (t_h) does not impact the critical path

Some other worked examples

Q2 S1 16/17 (a) The data flow graph (DFG) for a simple gradient calculation used in medical imaging is shown in Figure Q2. The algorithm consists of just 3 operations, addition, subtraction and the multiplication of a value with itself. The output of the last addition is truncated to the 16 least significant bits of the result of that addition. The non-pipelined module declaration for the *gradient* algorithm is given below:

```
module gradient (input [15:0] I0, I1, I2, I3, I4, output reg [15:0] O);
```

A 16-bit adder-subtractor has a propagation delay of 1ns, a 32-bit adder has propagation delay of 1.8ns and a 16-bit multiplier has a propagation delay of 3ns.



(i) Sketch the pipeline register locations to maximize throughput, and determine the maximum operating frequency, and the throughput and latency at that frequency.

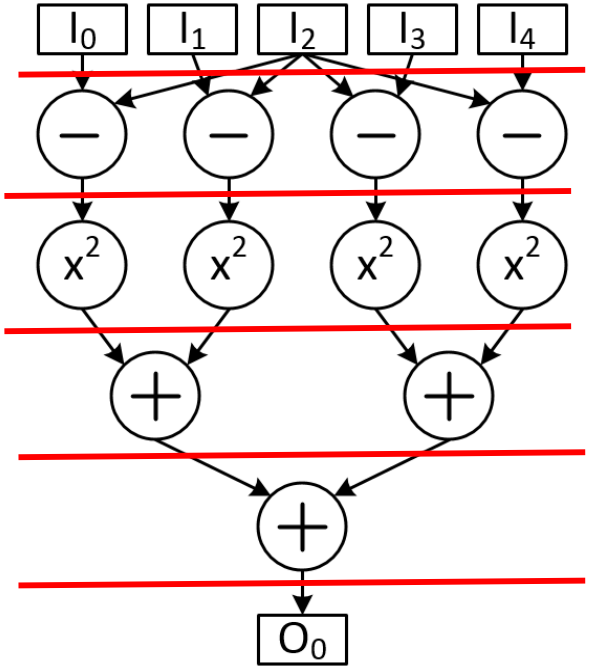
(6 marks)

ANS (a)(i) For maximum throughput, place a register stage after each set of operations (as shown). Then the maximum frequency is determined by the propagation delay through the multiplier (3ns) and is:

Frequency: 1/3ns = 333.33MHz

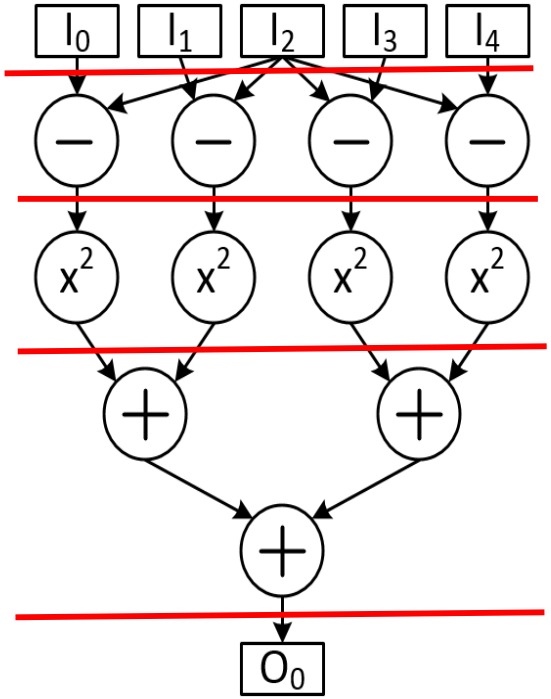
Throughput: one set of samples every 3 ns

Latency: 12ns



(ii) The results of Q2(a)(i) indicate that the throughput significantly exceeds the requirements, but the latency is too large and needs to be reduced by at least 3.5ns. Sketch the new pipeline register locations which give the highest throughput within the latency constraint. What is the new maximum operating frequency, throughput and latency?

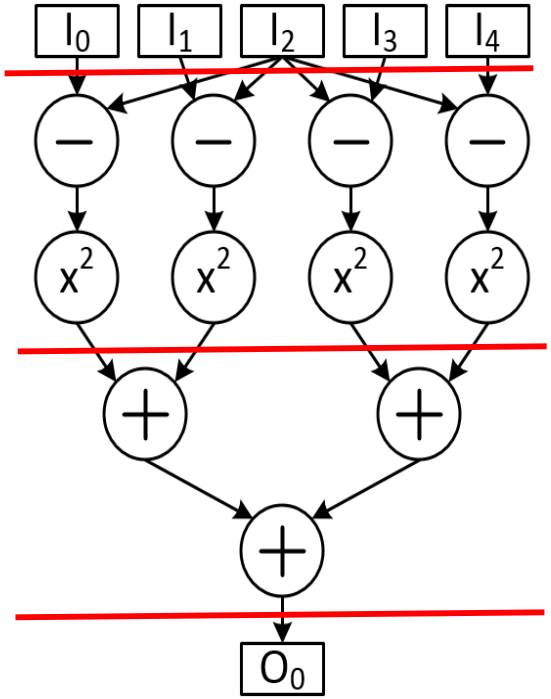
(5 marks)



ANS: New latency should be less than: 12-3.5 = 8.5ns
As a first guess, remove the register stage between the 32-bit adders. The new throughput is one set of samples every 3.6ns (278MHz). However, the latency is 10.8ns which is still too high.

(ii) The results of Q2(a)(i) indicate that the throughput significantly exceeds the requirements, but the latency is too large and needs to be reduced by at least 3.5ns. Sketch the new pipeline register locations which give the highest throughput within the latency constraint. What is the new maximum operating frequency, throughput and latency?

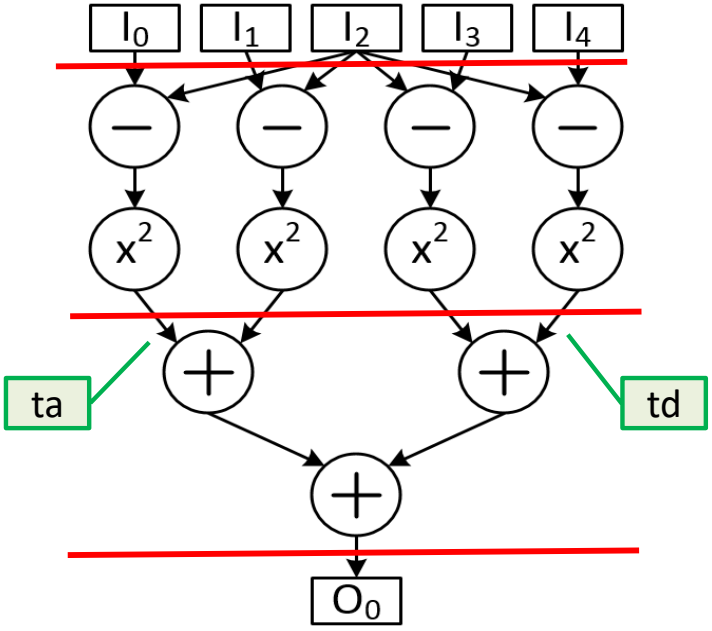
(5 marks)



ANS: New latency should be less than: 12-3.5 = 8.5ns
As a first guess, remove the register stage between the 32-bit adders. The new throughput is one set of samples every 3.6ns (278MHz). However, the latency is 10.8ns which is still too high.

As a second guess, also remove the register stage between the 16-bit subtractors and the multipliers (as shown). The new throughput is one set of samples every 4ns (250MHz). The new latency is 8ns which is 4ns less than the original version. So OK.

(iii) Write the Verilog code to implement the pipelined version of the gradient algorithm determined in Q2(a)(ii). Note: Ignore adder overflow and assume I_2 is always subtracted from the other inputs. All inputs are latched in a previous circuit and do not need to be latched inside the *gradient* module.

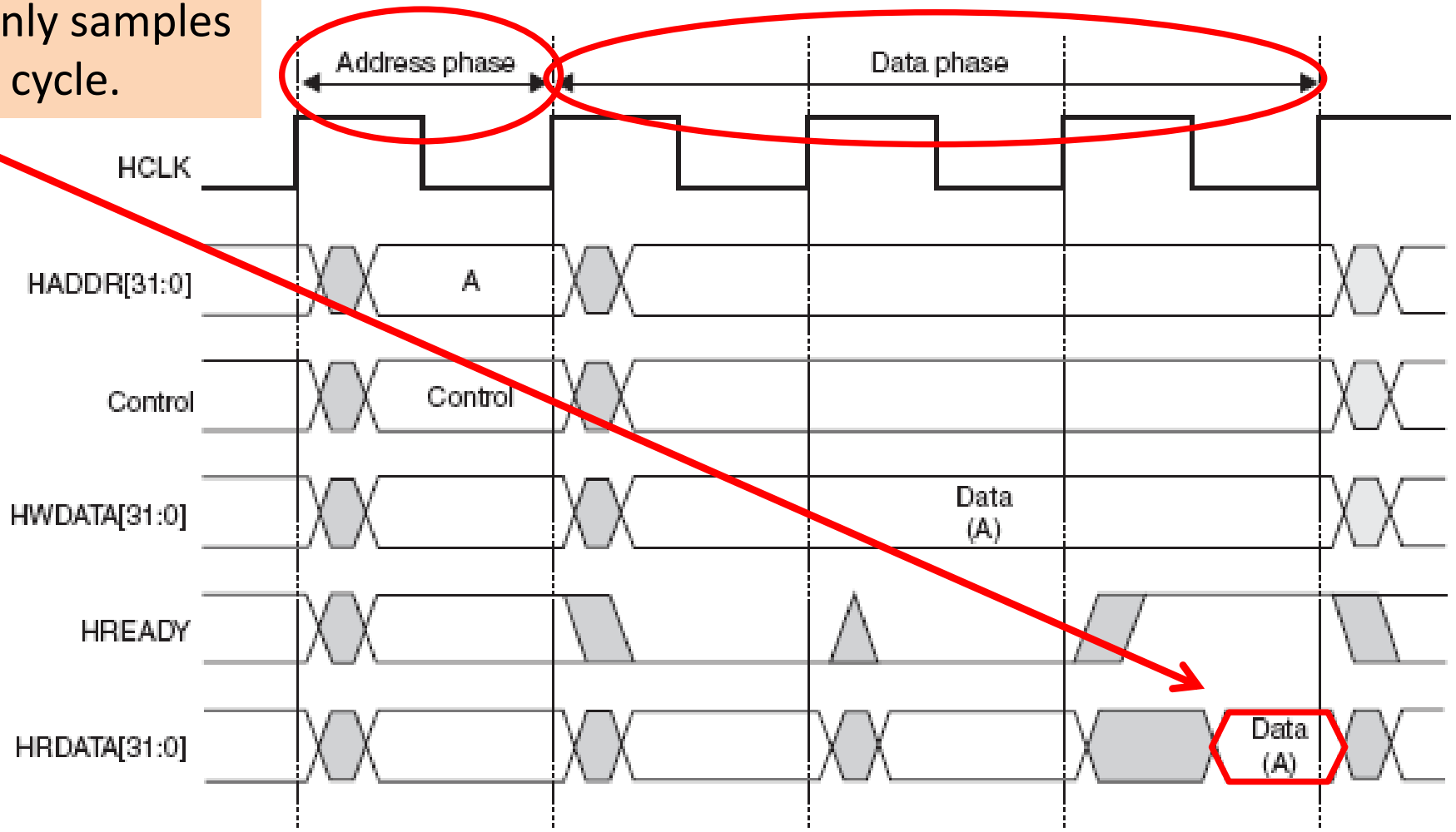



```
module gradient (input clk, rst, input [15:0] I0, I1,
                                 I2, I3, I4, output reg [15:0] O);
        reg [31:0] ta,tb,tc,td;
        wire [31:0] tadd;
        always @ (posedge clk)
                if (rst) begin
                        ta <= 0;
                        tb <= 0;
                        tc <= 0;
                        td <= 0;
                        O <= 0;
                end  else begin
                        ta <= (I0-I2)*(I0-I2);
                        tb <= (I1-I2)*(I1-I2);
                        tc <= (I3-I2)*(I3-I2);
                        td <= (I4-I2)*(I4-I2);
                        O <= tadd [15:0];
                end
        assign tadd = (ta+tb)+(tc+td);
endmodule
```

End of EC2 (Mod 5 & 7)

AMBA Transfer with wait states

Master only samples at end of cycle.



A slave may insert wait states into any transfer which extends the transfer allowing additional time for completion

Synchronous Timing

- The input (*d*) must be stable from t_s before, through to t_h after the rising edge of *clk*
- The output (*q*) will emerge t_{cQ} after the rising edge of *clk*

